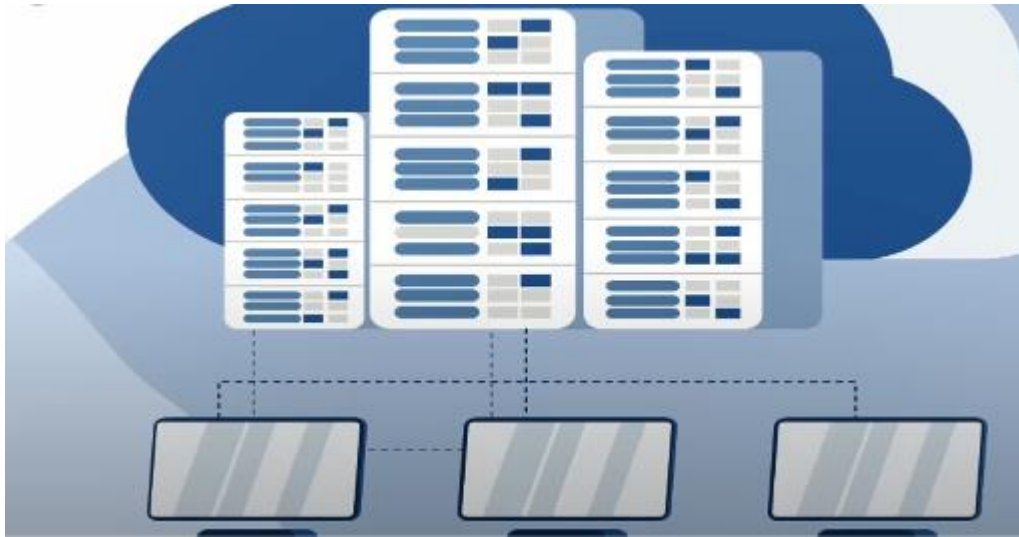


MICROSERVICES ARCHITECTURE

MICROSERVICES ARCHITECTURE

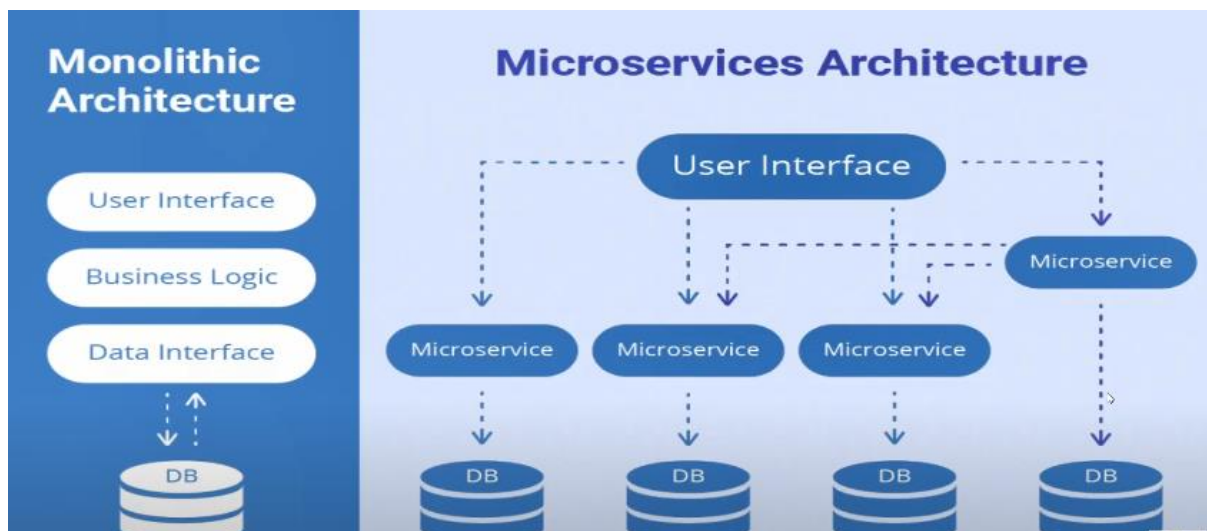


What are Microservices? Microservices are the architectural approach to build applications from small to large scale applications. With this architectural approach, an application is broken down into the smallest components, independent of each other. Unlike Monolithic architecture, where all the functionalities are targeted to build into a single project/application, Microservices helps to separate functionalities to develop in a more modular way and all modules work together to accomplish the specific targeted tasks. Need of a Microservice This architectural approach to developing software gives more modularity, being lightweight, and the ability to share similar functionalities across multiple applications. It is a major way of designing and optimizing app development towards a cloud-native model. To develop quality software, the faster development process for change in existing software features, easy to tackle any runtime issues after deployment into production. As we have challenges in every positive idea, Microservices also have the challenges like managing them aligning them to work without downtimes, because we maintain them in different servers and have to deal with the data and databases they use for transactions. Understanding API Gateway (Ocelot Gateway with example) API Gateway is nothing but a middleware layer of directing incoming HTTP request calls from Client applications to specific Microservice without directly exposing the Microservice details to the Client and returning the responses generated from the respective Microservice. Ocelot is an Open-Source API Gateway for the .NET/Core Platform which is officially supported by Microsoft. What it does is simple. It mimics masking multiple microservices existing behind that the client does not have to worry about the location of each and every Microservice.



Distributed System

Monolithic Architecture Vs MicroService Architecture



Monolithic architectures are dependent or compound development projects for user interface (UI), business functional component & data interface, whereas microservices, on the other hand, consist of multiple services developed for single purposes and which are independent of each other.

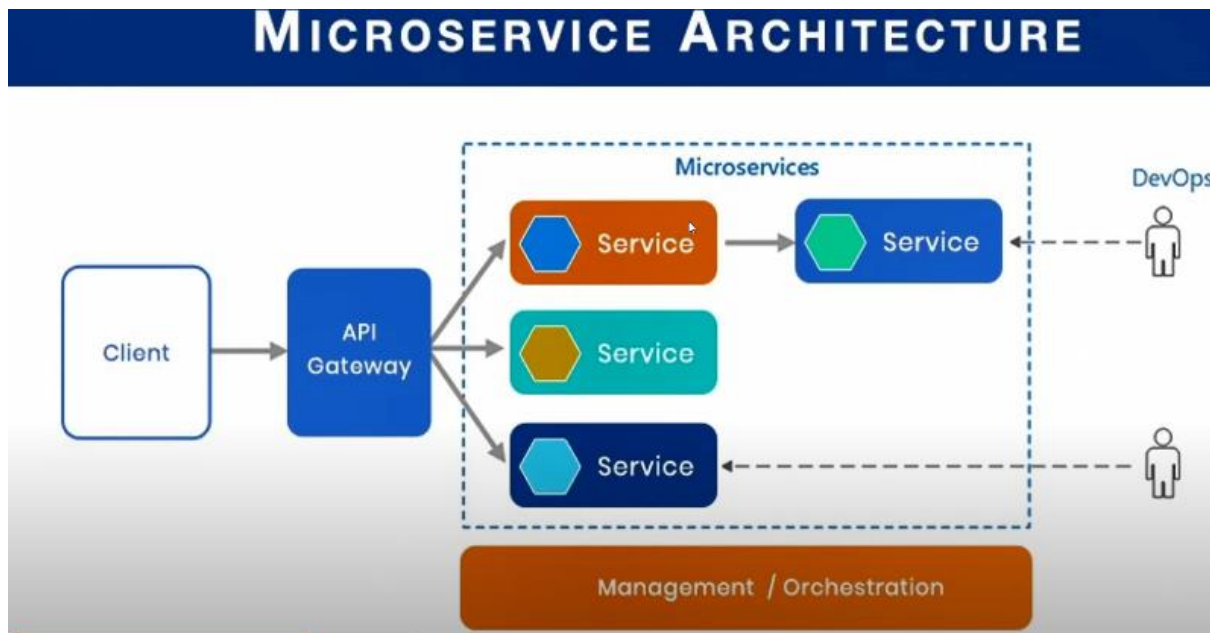
1. Microservices increase efficiency and decrease the down time.
2. Microservices have their own database coupled.

Benefit of MicroServices Are

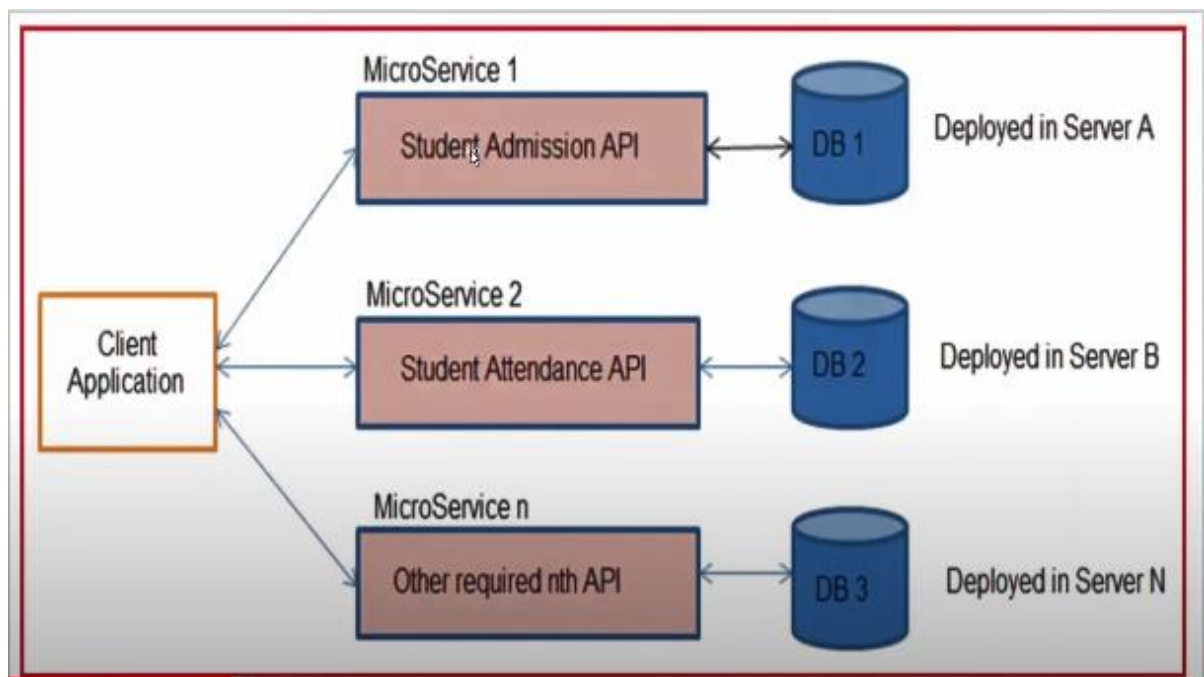
ADVANTAGES OF MICROSERVICES



1. Parellel work with team by multiple services.
2. Easy integrate the new functionality.
3. Independency run new module will not affect.
4. Downtime is nullable
5. Enhancement and development works on Agile Methodology.
6. Language independent for development.
7. Each microservice have its own database and increase efficiency.



API gateway helps to provide the abstraction level between client and API services. Ocelot provide us the api virtualization and which api gateway act as single point of entry.



Example how to achieve above

Demonstration :

A. Create Web API Project from visual Studio on .Net core 7.0

Solution Named :

MicroServices_MainApp

- APIProjectName : StudentAdmissionManagement
- APIProjectName : StudentAttendaneManagement
- OcelotGateway ProjectName : StudentApiGateway

Create model in StudentAdmissionManagement

```
using System.Diagnostics.Contracts;

namespace StudentAdmissionManagement.Model
{
    public class StudentAdmissionModel
    {
        // Declaring the properties of the student Attendance Model
        public int StudentId { get; set; }
        public string? StudentName { get; set; }
        public string? StudentClass { get; set; }
        public DateTime DateofJoining { get; set; }
    }
}
```

Work on Controller to create Get Method

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using StudentAdmissionManagement.Model;
using System.Globalization;

namespace StudentAdmissionManagement.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentAdmissionController : ControllerBase
    {
        //GET
        public IEnumerable<StudentAdmissionModel> Get()
        {
            StudentAdmissionModel studentadmissionmodel1 = new
StudentAdmissionModel();
            StudentAdmissionModel studentadmissionmodel2 = new
StudentAdmissionModel();

            studentadmissionmodel1.StudentId = 1;
            studentadmissionmodel1.StudentName = "Ankit";
            studentadmissionmodel1.StudentClass = "B.tech";
            studentadmissionmodel1.DateofJoining = DateTime.Now;

            studentadmissionmodel2.StudentId = 2;
            studentadmissionmodel2.StudentName = "Rohan";
        }
    }
}
```

```
        studentadmissionmodel2.StudentClass = "B.Ed";
        studentadmissionmodel2.DateofJoining = DateTime.Now;

        List<StudentAdmissionModel> studentAttendanceModelslist = new
List<StudentAdmissionModel> { studentadmissionmodel1, studentadmissionmodel2 };

        return studentAttendanceModelslist;
    }
}
```

Create Model in StudentAttendaneManagement

```
namespace StudentAttendaneManagement.Model
{
    public class StudentAttendanceDetailsModel
    {
        public int StudentId { get; set; }
        public string? StudentName { get; set; }
        public double AttendancePercentage { get; set; }
    }
}
```

Work On Controller :

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using StudentAttendaneManagement.Model;

namespace StudentAttendaneManagement.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentAttendanceController : ControllerBase
    {
        //GET
        [HttpGet]

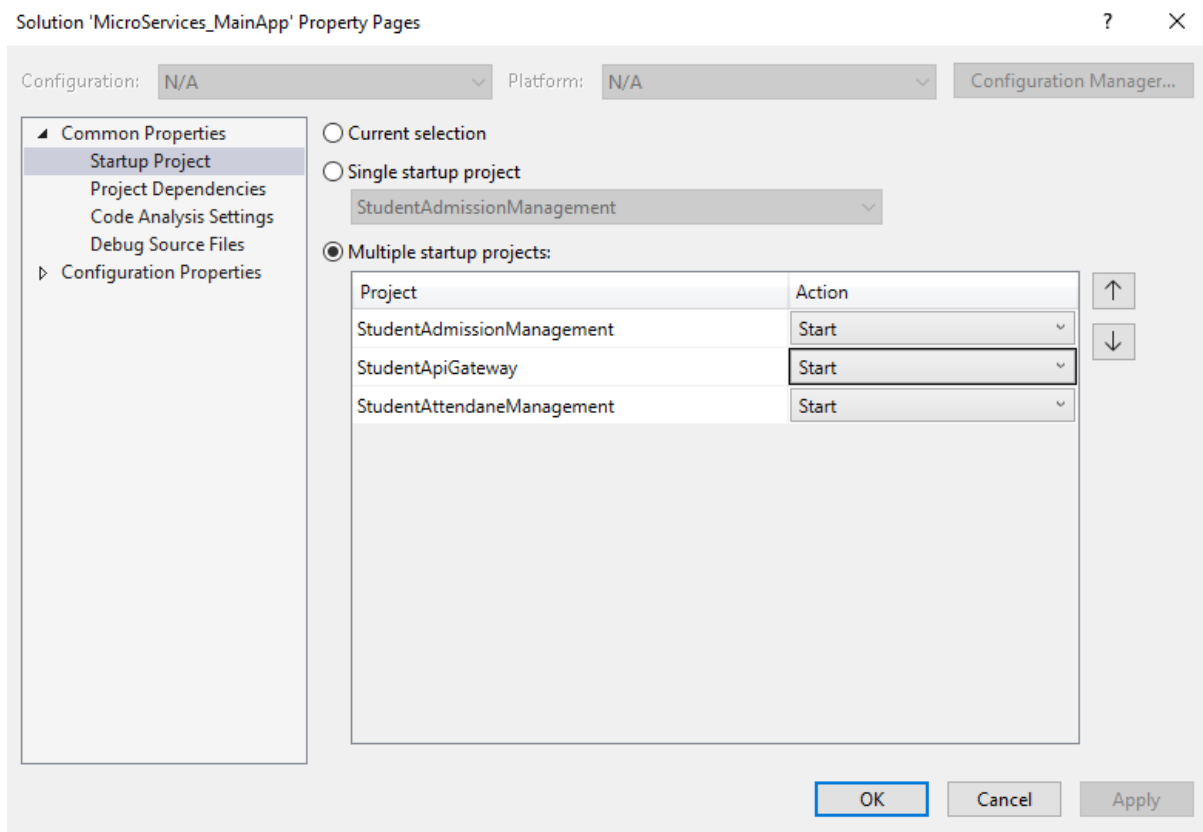
        public IEnumerable<StudentAttendanceDetailsModel> Get()
        {
            StudentAttendanceDetailsModel attendanceDetailobj1= new
StudentAttendanceDetailsModel();
            StudentAttendanceDetailsModel attendanceDetailobj2 = new
StudentAttendanceDetailsModel();

            attendanceDetailobj1.StudentId =1;
            attendanceDetailobj1.StudentName = "Ankit";
            attendanceDetailobj1.AttendancePercentage = 100.00;

            attendanceDetailobj2.StudentId = 2;
            attendanceDetailobj2.StudentName = "Rohan";
            attendanceDetailobj2.AttendancePercentage = 50.00;

            List<StudentAttendanceDetailsModel> objlist =new
List<StudentAttendanceDetailsModel> { attendanceDetailobj1,attendanceDetailobj2
};
            return objlist;
        }
    }
}
```

RightClick go to Properties : change the startup Action set to star All



3. Project Create json file to configure the routes for ocelot calling.
StudentApiGateway

ocelot.json

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/StudentAttendance",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 7024
        }
      ],
      "UpstreamPathTemplate": "/ApiGatewayOcelot/Attendance",
      "UpstreamHttpMethod": [ "Get", "Post" ]
    },
    {
      "DownstreamPathTemplate": "/api/StudentAdmission",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 7209
        }
      ]
    }
  ]
}
```

```
    ],  
    "UpstreamPathTemplate": "/ApiGatewayOcelot/Admission",  
    "UpstreamHttpMethod": [ "Get", "Post" ]  
  }  
]  
}
```

Routes : it's define the api route and its configuration

DownstreamPathTemplate is the actual api url

DownstreamScheme : protocol http/https

downstreamHostAndPorts [{"Host": "", "Port": ""}]

UpstreamPathTemplate : to define the url for routing from gateway

It's exposed to the client as apigateway which will call the actual api

UpstreamHttpMethod define the type of calls/method allowed

In the third APIgateway Project need to install the Nuget Package

Ocelot

Browse Installed Updates

ocelot x ↻ ☐ Include prerelease



Ocelot by Tom Pallister, 9.73M downloads
Ocelot is an API gateway based on .NET stack.

23.3.3

Configure the ocelot services in the program.cs

```
//step 1  
builder.Configuration.SetBasePath(builder.Environment.ContentRootPath).AddJsonFil  
e("ocelot.json", optional:false, reloadOnChange: true);  
.....
```

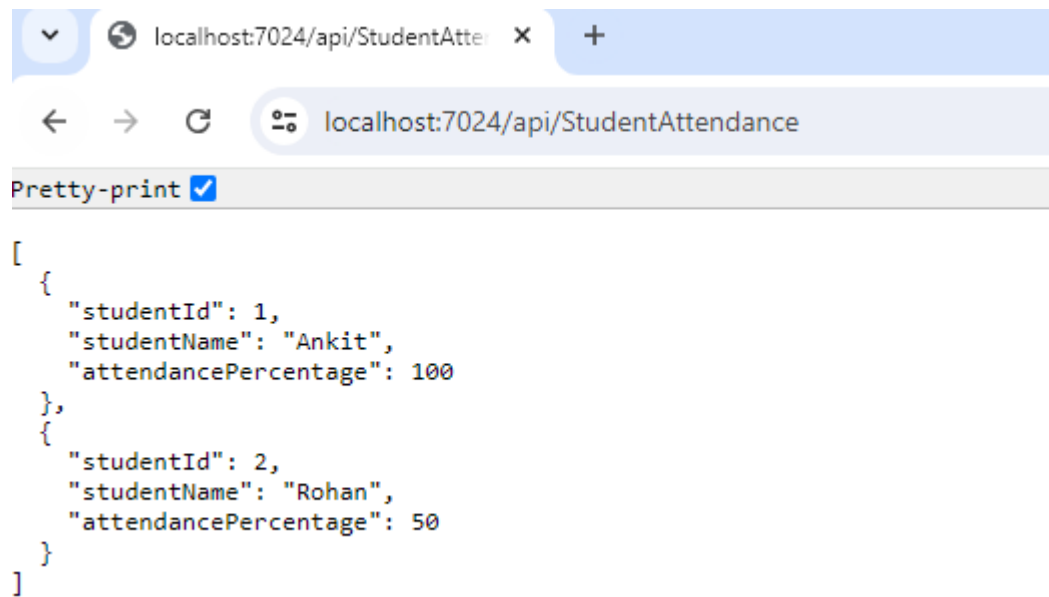
Note filename should be same as we created for route configuration

.....

```
//step 2  
builder.Services.AddOcelot();  
//step 3  
app.UseOcelot().Wait();
```

Run All projects together let see the demonstration

Project 1:

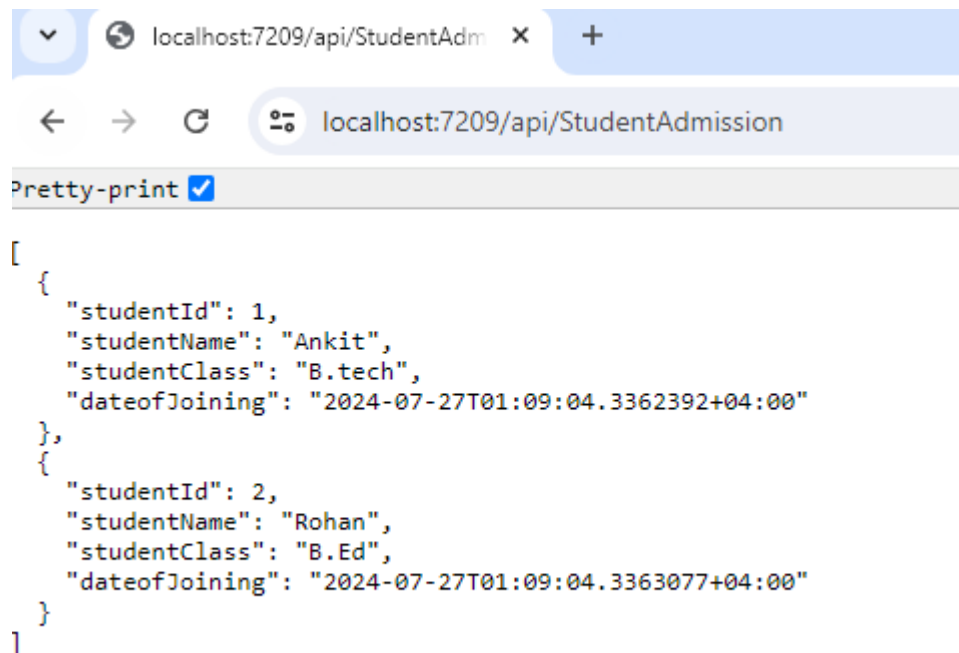


localhost:7024/api/StudentAttendance

Pretty-print ☒

```
[
  {
    "studentId": 1,
    "studentName": "Ankit",
    "attendancePercentage": 100
  },
  {
    "studentId": 2,
    "studentName": "Rohan",
    "attendancePercentage": 50
  }
]
```

Project 2

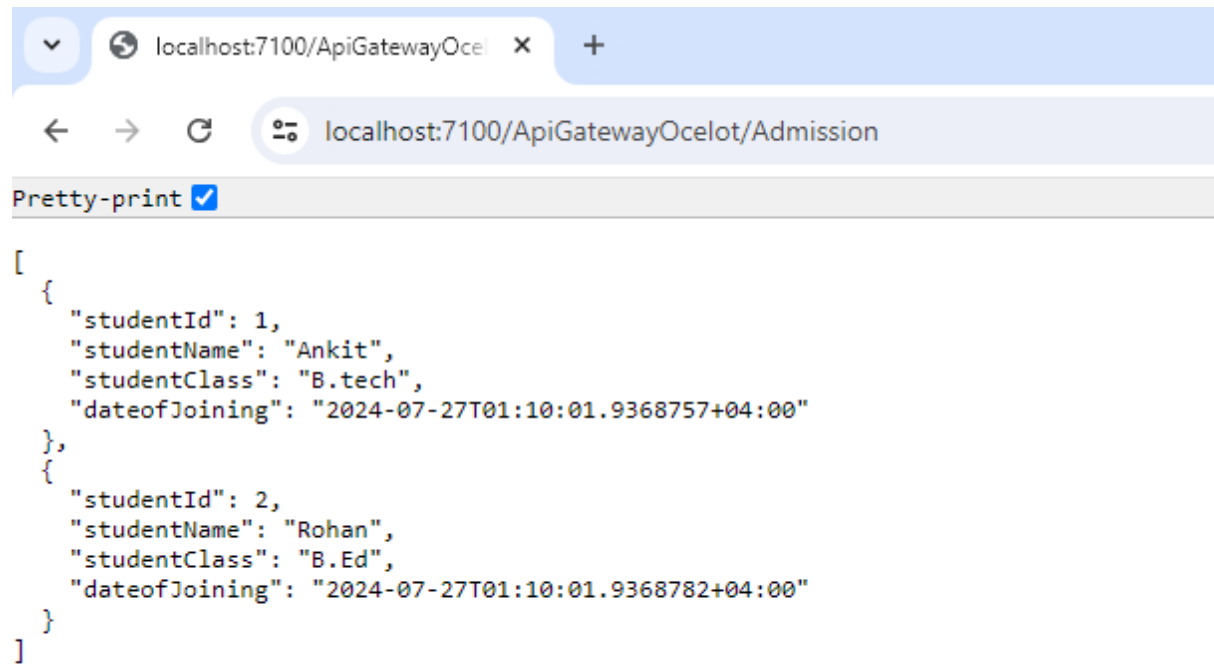


localhost:7209/api/StudentAdmission

Pretty-print ☒

```
[
  {
    "studentId": 1,
    "studentName": "Ankit",
    "studentClass": "B.tech",
    "dateofJoining": "2024-07-27T01:09:04.3362392+04:00"
  },
  {
    "studentId": 2,
    "studentName": "Rohan",
    "studentClass": "B.Ed",
    "dateofJoining": "2024-07-27T01:09:04.3363077+04:00"
  }
]
```

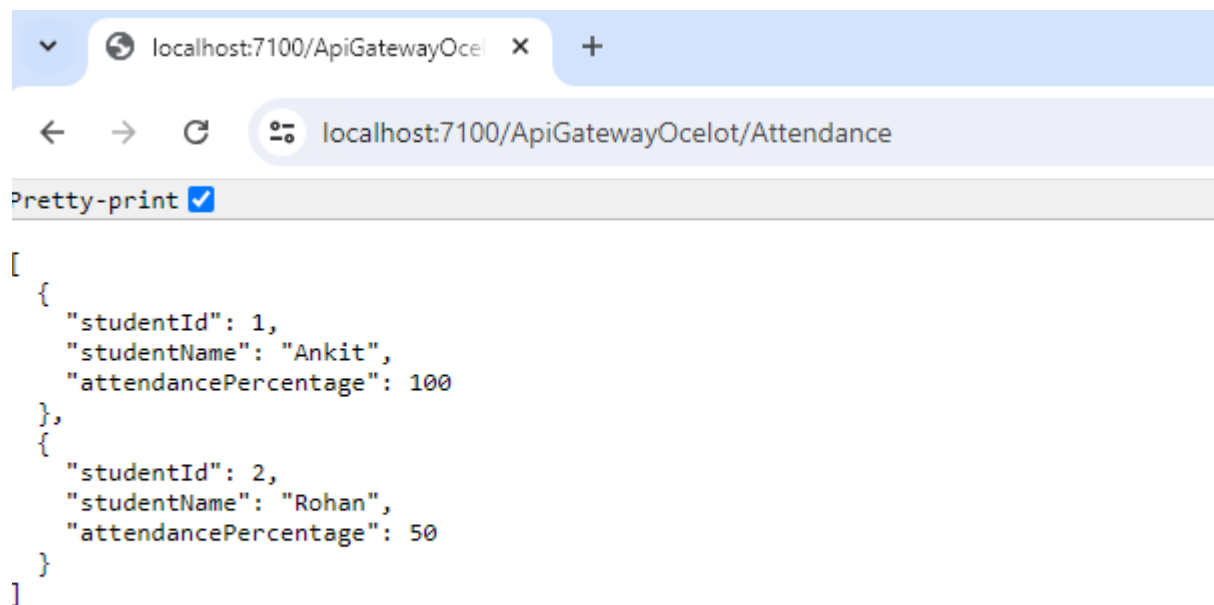
GateWay Project



The screenshot shows a web browser window with the address bar displaying `localhost:7100/ApiGatewayOcelot/Admission`. Below the address bar, there is a "Pretty-print" checkbox which is checked. The main content area displays a JSON array of two student objects. The first object has `studentId`: 1, `studentName`: "Ankit", `studentClass`: "B.tech", and `dateofJoining`: "2024-07-27T01:10:01.9368757+04:00". The second object has `studentId`: 2, `studentName`: "Rohan", `studentClass`: "B.Ed", and `dateofJoining`: "2024-07-27T01:10:01.9368782+04:00".

```
[
  {
    "studentId": 1,
    "studentName": "Ankit",
    "studentClass": "B.tech",
    "dateofJoining": "2024-07-27T01:10:01.9368757+04:00"
  },
  {
    "studentId": 2,
    "studentName": "Rohan",
    "studentClass": "B.Ed",
    "dateofJoining": "2024-07-27T01:10:01.9368782+04:00"
  }
]
```

`"UpstreamPathTemplate": "/ApiGatewayOcelot/Admission",`



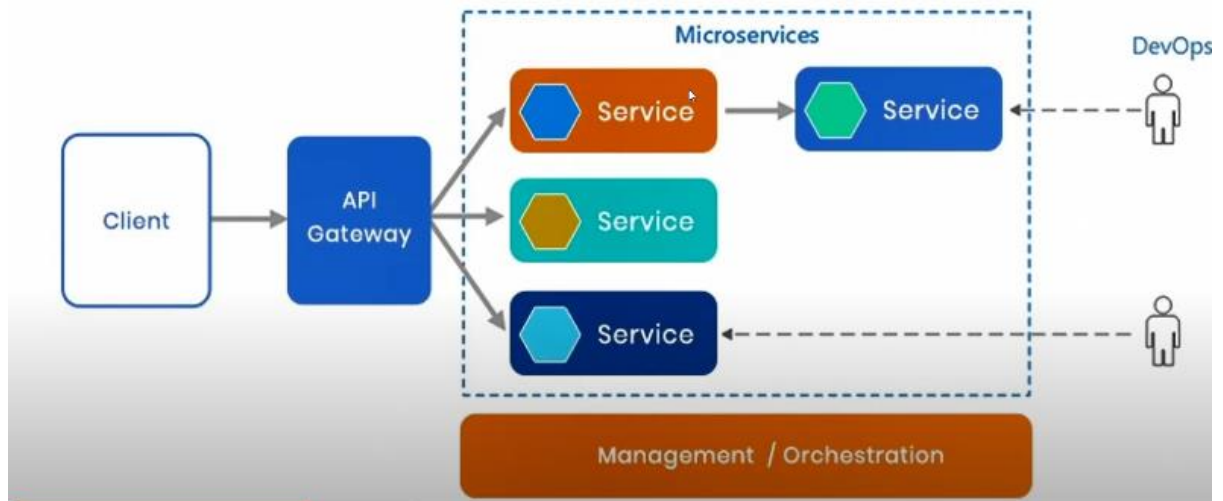
The screenshot shows a web browser window with the address bar displaying `localhost:7100/ApiGatewayOcelot/Attendance`. Below the address bar, there is a "Pretty-print" checkbox which is checked. The main content area displays a JSON array of two student objects. The first object has `studentId`: 1, `studentName`: "Ankit", and `attendancePercentage`: 100. The second object has `studentId`: 2, `studentName`: "Rohan", and `attendancePercentage`: 50.

```
[
  {
    "studentId": 1,
    "studentName": "Ankit",
    "attendancePercentage": 100
  },
  {
    "studentId": 2,
    "studentName": "Rohan",
    "attendancePercentage": 50
  }
]
```

`"UpstreamPathTemplate": "/ApiGatewayOcelot/Attendance",`

So based on above url which should expose to the client doesnot know the actual name or implement , deployment of the microservices or services api.

MICROSERVICE ARCHITECTURE



Api gateway provides the interface level layer to gives the abstraction layer to achieve the security and bunch of the services access together on the single point of contact.